

Advanced Databases

Partitioning Proposition

Marek Kocik 273102

Jan Kański 264694

May 2026

1 Partitioning Proposition

The baseline workload analysis revealed that primary performance bottlenecks are caused by full table scans and massive aggregations spilling to disk. The proposed partitioning strategy is designed to mitigate these high-cost operations identified in the baseline execution plans.

Table 1: Primary Performance Bottlenecks from Baseline Analysis

Operation in Query Plan	Target Table	Bottleneck Details
HASH GROUP BY / SORT	REVIEWS	Q3 (Cost: 157K, TempSpc: 322MB)
TABLE ACCESS FULL	CALENDAR	Q6 (Cost: 6685)
DELETE	CALENDAR	Q7 (Cost: 148K)
TABLE ACCESS FULL	LISTINGS	Q1, Q3, Q4 (Cost: 3568)

1.1 Proposed Partitioning Schemes

We propose the following distinct partitioning methods, each targeting a specific operation

1.1.1 RANGE Partitioning on calendar

We propose partitioning the 3.7-million-row `calendar` table by `listing_date` using monthly or yearly RANGE intervals.

Targeted Operations:

- **DELETE and UPDATE on CALENDAR (in Q6, Q7):** The goal is to replace costly row-by-row deletions (Cost: 148K in Q7) and also full table scans (Cost: 6685 in Q6) with targeted operations. Partitioning allows an entire segment of old data to be dropped via partition pruning.

```
CREATE TABLE calendar ( ... )
PARTITION BY RANGE (listing_date) (
    PARTITION p_before_2024 VALUES LESS THAN (TO_DATE('2024-01-01', 'YYYY-MM-DD')),
    PARTITION p_2024 VALUES LESS THAN (TO_DATE('2025-01-01', 'YYYY-MM-DD')),
    PARTITION p_2025 VALUES LESS THAN (TO_DATE('2026-01-01', 'YYYY-MM-DD'))
);
```

Listing 1: RANGE partitioning for the `calendar` table.

Q7

Id	Operation	Name	Rows	Bytes	TempSpc	Cost	(%CPU)	Time
0	DELETE STATEMENT		1	39		148K	(1)	00:00:06
1	DELETE	CALENDAR	1					
2	HASH JOIN SEMI		1	39		148K	(1)	00:00:06
3	TABLE ACCESS FULL	CALENDAR	1	26		6731	(2)	00:00:01
4	VIEW	VW_NSO_1	1	13		141K	(1)	00:00:06

* 5	FILTER								
6	SORT GROUP BY			1	111			141K	(1) 00:00:06
* 7	HASH JOIN RIGHT OUTER			633K	67M			141K	(1) 00:00:06
* 8	TABLE ACCESS FULL	CALENDAR		27726	703K			6682	(2) 00:00:01
* 9	HASH JOIN OUTER			597K	48M	7008K		135K	(1) 00:00:06
* 10	HASH JOIN OUTER			115K	5649K	2160K		5120	(1) 00:00:01
* 11	HASH JOIN			58117	1475K			4319	(1) 00:00:01
* 12	TABLE ACCESS FULL	HOSTS		34579	337K			751	(1) 00:00:01
13	TABLE ACCESS FULL	LISTINGS		83208	1300K			3568	(1) 00:00:01
* 14	TABLE ACCESS FULL	LISTING_SCRAPE_SNAPSHOT		164K	3855K			415	(2) 00:00:01
* 15	TABLE ACCESS FULL	REVIEWS		346K	11M			128K	(1) 00:00:06

Q6

Id	Operation	Name	Rows	Bytes	TempSpc	Cost (%CPU)	Time	
0	UPDATE STATEMENT		61014	1966K		136K (1)	00:00:06	
1	UPDATE	CALENDAR						
* 2	HASH JOIN SEMI		61014	1966K	1912K	136K (1)	00:00:06	
* 3	TABLE ACCESS FULL	CALENDAR	61014	1191K		6685 (2)	00:00:01	
4	VIEW	VW_NS0_1	122K	1549K		129K (1)	00:00:06	
* 5	FILTER							
* 6	HASH JOIN ANTI		122K	4528K	3344K	129K (1)	00:00:06	
* 7	TABLE ACCESS FULL	LISTING_SCRAPE_SNAPSHOT	122K	1908K		410 (1)	00:00:01	
* 8	TABLE ACCESS FULL	REVIEWS	346K	7446K		128K (1)	00:00:06	

1.1.2 HASH Partitioning on reviews

We propose partitioning the massive `reviews` table (approx. 6 million rows) by `HASH` on its `listing_id` column into a power-of-two number of partitions (e.g., 16 or 32).

Targeted Operations:

- **HASH GROUP BY and SORT GROUP BY (in Q3, Q5):** The baseline execution plan for Q3 demonstrates a massive `HASH GROUP BY` operation (Cost: 157K) forcing 322MB of data to spill to the temporary disk space (TempSpc), significantly degrading performance (Time: 00:00:07). By partitioning the table via `HASH(listing_id)`, the database can split this massive aggregation into smaller, parallel processes.

```
CREATE TABLE reviews ( ... )
PARTITION BY HASH (listing_id) PARTITIONS 8;
```

Listing 2: HASH partitioning for the `reviews` table.

Id	Operation	Name	Rows	Bytes	TempSpc	Cost (%CPU)	Time	
0	SELECT STATEMENT		121	11132		182K (1)	00:00:08	
1	TEMP TABLE TRANSFORMATION							
2	LOAD AS SELECT (CURSOR DURATION MEMORY)	SYS_TEMP_OFD9D664F_7811F0						
3	HASH GROUP BY		49508	2175K	322M	157K (1)	00:00:07	
4	MERGE JOIN		5982K	256M		132K (1)	00:00:06	
5	SORT JOIN		5982K	148M		128K (1)	00:00:06	
6	VIEW	VW_GBC_9	5982K	148M		128K (1)	00:00:06	
7	HASH GROUP BY		5982K	74M		128K (1)	00:00:06	
8	TABLE ACCESS FULL	REVIEWS	5982K	74M		128K (1)	00:00:06	
* 9	SORT JOIN		83208	1543K	5240K	4059 (1)	00:00:01	
10	VIEW	VW_GBF_10	83208	1543K		3568 (1)	00:00:01	
11	TABLE ACCESS FULL	LISTINGS	83208	1300K		3568 (1)	00:00:01	
12	WINDOW SORT		121	11132		24878 (1)	00:00:01	
* 13	HASH JOIN		121	11132		24793 (1)	00:00:01	
14	JOIN FILTER CREATE	:BF0000	122	9638		3779 (1)	00:00:01	
* 15	HASH JOIN		122	9638		3779 (1)	00:00:01	
16	NESTED LOOPS		124	7316		3695 (1)	00:00:01	
17	NESTED LOOPS		124	7316		3695 (1)	00:00:01	
18	VIEW		124	4092		3571 (1)	00:00:01	
* 19	FILTER							
20	HASH GROUP BY		124	2480		3571 (1)	00:00:01	
21	TABLE ACCESS FULL	LISTINGS	83208	1625K		3568 (1)	00:00:01	
* 22	INDEX UNIQUE SCAN	HOSTS_PK	1			0 (0)	00:00:01	
23	TABLE ACCESS BY INDEX ROWID	HOSTS	1	26		1 (0)	00:00:01	
* 24	VIEW		49508	966K		84 (0)	00:00:01	
25	TABLE ACCESS FULL	SYS_TEMP_OFD9D664F_7811F0	49508	966K		84 (0)	00:00:01	
26	SORT AGGREGATE		1	13				
27	VIEW		49508	628K		84 (0)	00:00:01	
28	TABLE ACCESS FULL	SYS_TEMP_OFD9D664F_7811F0	49508	966K		84 (0)	00:00:01	
29	VIEW		49508	628K		21014 (1)	00:00:01	
30	HASH GROUP BY		49508	2127K	3336K	21014 (1)	00:00:01	
31	JOIN FILTER USE	:BF0000	65272	2804K		20397 (1)	00:00:01	
* 32	HASH JOIN		65272	2804K	2280K	20397 (1)	00:00:01	
33	TABLE ACCESS FULL	LISTINGS	83208	1300K		3568 (1)	00:00:01	
34	VIEW	VW_GBC_16	65272	1784K		16595 (1)	00:00:01	
35	HASH GROUP BY		65272	1593K	130M	16595 (1)	00:00:01	
* 36	TABLE ACCESS FULL	CALENDAR	3785K	90M		6643 (1)	00:00:01	

1.1.3 LIST Partitioning on listings

We propose partitioning the `listings` table (83,000 rows) using the LIST method on the `room_type` column.

Targeted Operation and Architectural Justification:

- **TABLE ACCESS FULL (in Q1, Q3, Q4):** The optimizer repeatedly performs full table scans on `LISTINGS` (Cost: 3568) before filtering records. LIST partitioning enables **Partition Pruning**. If the application searches specifically for 'Entire home/apt', the database will completely bypass irrelevant segments, reducing I/O cost to a fraction of the baseline.
- **Avoidance of Segment Explosion:** While a HASH or Composite strategy on `listings` could theoretically enable a Partition-Wise Join with `reviews`, it would fragment this relatively small table into hundreds of micro-segments, degrading performance. Instead, by relying on LIST pruning, the filtered subset of data (e.g., just 15,000 rows) will easily load completely into memory, allowing a standard HASH JOIN with `reviews` to execute near-instantaneously.

```
CREATE TABLE listings ( ... )  
PARTITION BY LIST (room_type) (  
    PARTITION p_entire_home VALUES ('Entire home/apt'),  
    PARTITION p_private_room VALUES ('Private room'),  
    PARTITION p_other VALUES (DEFAULT)  
);
```

Listing 3: LIST partitioning for the `listings` table.

1.2 Experimental Comparisons

1.2.1 Comparison 1: Partition Granularity for the calendar Table

The goal is to quantify the trade-off between finer-grained pruning for queries spanning less than a year (using 6-month or monthly intervals) and the increased management overhead of maintaining more partitions compared to yearly intervals.

1.2.2 Comparison 2: RANGE vs LIST on Dates

This experiment is designed to confirm the theoretical superiority of RANGE for handling date-range predicates (e.g., `date >= constant`) by comparing it to LIST partitioning on a virtual column for the year. We hypothesize that LIST will prove inefficient for such operations.